Machine Learning and Deep Learning
Lecture-13

By: Somnath Mazumdar
Assistant Professor
sma.digi@cbs.dk

Spring 2026

Overview

- Hyper-parameter Optimization (HPO)
- Generative Adversarial Network (GAN)

Hyper-parameter Optimization (HPO)

Hyper-parameters

- Neural networks have several **hyperparameters**, such as learning rate and regularization strength, that must be set before training.
- Hyperparameters control the **design and training behavior** of the model and are different from the learned parameters (weights and biases).
- Model parameters (weights) are optimized with backpropagation and gradient-based optimizers (e.g., SGD, Adam) for a fixed choice of hyperparameters.
- Hyperparameters should not be tuned on the same data used only to fit the weights.
- A portion of the data is held out as a **validation set**.
- Procedure: train on the training set with different hyperparameter settings, evaluate on the validation set, and select the best configuration.
- This helps avoid overfitting to the training data and leads to better generalization.

Challenges Hyper-parameter Optimization

- Function evaluations can be extremely expensive for large models (e.g., deep networks), complex ML pipelines, or large datasets.

- The configuration space is often complex and high-dimensional, mixing continuous, categorical, and conditional hyperparameters; it is not always clear which ones matter most.

- There is no direct access to gradients of the hyperparameter objective (e.g., validation loss), and classical assumptions like convexity or smoothness typically do not hold, so standard gradient-based optimizers are not applicable.

- We cannot directly optimize true generalization performance because we only observe it through finite training/validation sets; overfitting to the validation set (overtuning) is a real risk, especially with limited data.

F. Hutter et al. (eds.), Automated Machine Learning, The Springer Series on Challenges in Machine Learning, https://doi.org/10.1007/978-3-030-05318-5_1

Hyper-parameters

ML Algorithm	Main HPs	Optional HPs	HPO methods	Libraries
Linear regression	–	–	–	–
Ridge & lasso	alpha	–	BO-GP	Skpot
Logistic regression	penalty, c, solver	-	BO-TPE, SMAC	Hyperopt, SMAC
KNN	n_neighbors	weights, p, algorithm	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
SVM	C, kernel, epsilon (for SVR)	gamma, coef0, degree	BO-TPE, SMAC, BOHB	Hyperopt, SMAC, BOHB
NB	alpha	–	BO-GP	Skpot
DT	criterion, max_depth, min_samples_split, min_samples_leaf, max_features	splitter, min_weight_fraction_leaf, max_leaf_nodes	GA, PSO, BO-TPE, SMAC, BOHB	TPOT, Optunity, SMAC, BOHB
RF & ET	n_estimators max_depth, criterion, min_samples_split, min_samples_leaf, max_features	splitter, min_weight_fraction_leaf, max_leaf_nodes	GA, PSO, BO-TPE, SMAC, BOHB	TPOT, Optunity, SMAC, BOHB
XGBoost	n_estimators, max_depth, learning_rate, subsample, colsample_bytree,	min_child_weight, gamma, alpha, lambda	GA, PSO, BO-TPE, SMAC, BOHB	TPOT, Optunity, SMAC, BOHB
Voting	estimators, voting	weights	GS	Sklearn
Bagging	base_estimator, n_estimators	max_samples, max_features	GS, BOs	sklearn, Skpot, Hyperopt, SMAC
AdaBoost	base_estimator, n_estimators, learning_rate	-	BO-TPE, SMAC	Hyperopt, SMAC
Deep learning	number of hidden layers, 'units' per layer, loss, optimizer, Activation, learning_rate, dropout rate, epochs, batch_size, early stop patience	number of frozen layers (if transfer learning is used)	PSO, BOHB	Optunity, BOHB
K-means	n_clusters	init, n_init, max_iter	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
Hierarchical clustering	n_clusters, distance_threshold	linkage	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
DBSCAN	eps, min_samples	-	BO-TPE, SMAC, BOHB	Hyperopt, SMAC, BOHB
Gaussian mixture	n_components	covariance_type, max_iter, tol	BO-GP	Skpot
PCA	n_components	svd_solver	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband
LDA	n_components	solver, shrinkage	BOs, Hyperband	Skpot, Hyperopt, SMAC, Hyperband

Yang, Li, and Abdallah Shami. "On hyperparameter optimization of machine learning algorithms: Theory and practice." *Neurocomputing* 415 (2020): 295-316.

Parameter Initialization

- Parameter initialization is crucial for neural network performance.
- You cannot initialize all weights in the same layer to the same value (e.g., all zeros), or neurons will remain identical (symmetry problem).
- Weights that are too large can cause exploding activations or gradients; weights that are too small can cause vanishing gradients.
- A traditional “small random” initialization used to be:
  - Initialize bias terms to 0.
  - Initialize weights with small random values.
- Modern practice uses principled schemes, which scale the variance of the weights based on the number of input/output units in each layer.
- Initializing with deep unsupervised models (e.g., stacked autoencoders) was a popular approach more than 10 years ago, but is rarely needed now due to better initialization and optimizers.

Hyper-parameters

- How should candidate hyperparameters be selected for testing?
  - A simple approach is grid search, where you define a discrete set of values for each hyperparameter and evaluate all combinations.
  - The number of grid points grows exponentially with the number of hyperparameters (curse of dimensionality).
  - Example: with 5 hyperparameters and 10 candidate values for each, you need 10^5=100,000 training runs to evaluate all combinations.
  - Because of this exponential growth and the fact that many hyperparameters are not equally important, grid-based exploration is usually not the best choice in high-dimensional spaces; random search and Bayesian optimization are typically more efficient.

Hyper-parameters

- **Random Search for Hyperparameters**
  - Instead of evaluating every point on a grid, randomly sample hyperparameter configurations within the chosen ranges.
- Log-space sampling (for scale-sensitive hyperparameters)
  - For hyperparameters that vary over orders of magnitude (e.g., learning rate, regularization strength), it is better to sample their logarithms uniformly rather than the raw values.
  - Example: instead of sampling a learning rate αα uniformly between 0.1 and 0.001, sample $\log10(\alpha)$ uniformly between −1 and −3, then set $\alpha=10^{\log10(\alpha)}$.
  - Search for such hyperparameters in logarithmic space rather than linear space, to cover small and large values more evenly.

Hyper-parameter Optimization Techniques Comparisions

The comparison of common HPO algorithms (n is the number of hyper-parameter values and k is the number of hyper-parameters).

HPO Method	Strengths	Limitations	Time Complexity
GS	• Simple.	• Time-consuming • Only efficient with categorical HPs.	$O(n^k)$
RS	• More efficient than GS • Enable parallelization.	• Not consider previous results • Not efficient with conditional HPs.	$O(n)$
Gradient-based models	• Fast convergence speed for continuous HPs.	• Only support continuous HPs • May only detect local optimums.	$O(n^k)$
BO-GP	• Fast convergence speed for continuous HPs.	• Poor capacity for parallelization • Not efficient with conditional HPs.	$O(n^3)$
SMAC	• Efficient with all types of HPs.	• Poor capacity for parallelization.	$O(nlogn)$
BO-TPE	• Efficient with all types of HPs • Keep conditional dependencies.	• Poor capacity for parallelization.	$O(nlogn)$
Hyperband	• Enable parallelization.	• Not efficient with conditional HPs • Require subsets with small budgets to be representative.	$O(nlogn)$
BOHB	• Efficient with all types of HPs • Enable parallelization.	• Require subsets with small budgets to be representative.	$O(nlogn)$
GA	• Efficient with all types of HPs • Not require good initialization.	• Poor capacity for parallelization.	$O(n^2)$
PSO	• Efficient with all types of HPs • Enable parallelization.	• Require proper initialization.	$O(nlogn)$

Yang, Li, and Abdallah Shami. "On hyperparameter optimization of machine learning algorithms: Theory and practice." *Neurocomputing* 415 (2020): 295-316.

Hyper-parameter Optimization Frameworks

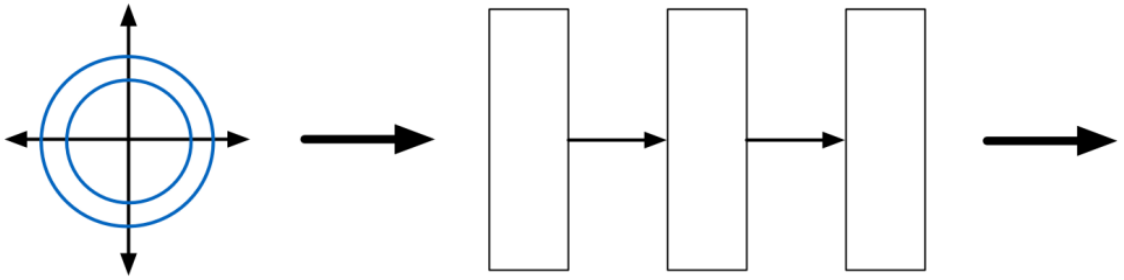
- Sklearn:
  - GridSearchCV implements grid search over a specified parameter grid to find good hyperparameters.
  - RandomizedSearchCV implements random search, sampling a fixed number of random hyperparameter combinations from given distributions.
- TensorFlow
  - Trieste: Bayesian optimization toolbox built on TensorFlow (https://github.com/secondmind-labs/trieste)

Yang, Li, and Abdallah Shami. "On hyperparameter optimization of machine learning algorithms: Theory and practice." *Neurocomputing* 415 (2020): 295-316.

Adversarial Examples: GAN

Generative Models

- Generative models learn a mapping from random noise vectors to things that look like, e.g., images.



Each dimension of the code vector is sampled independently from a simple distribution, e.g. Gaussian or uniform.

This is fed to a (deterministic) generator network.

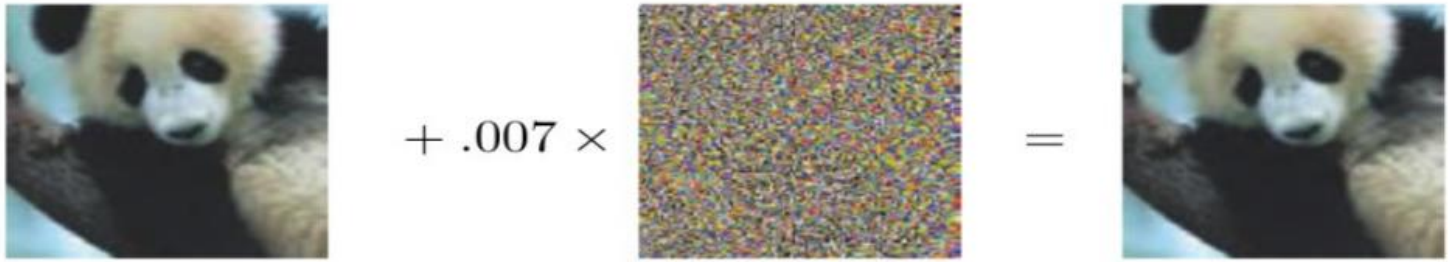
- Need to have some criterion for evaluating image quality, then compute gradient wrt to parameters,
- Update parameters for better image.

Putting into the context

Panda

Gibbon

Misclassification



$+ .007 \times$ $=$

$\boldsymbol{x}$

$y =$"panda"
w/ 57.7%
confidence

$\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$

"nematode"
w/ 8.2%
confidence

$\boldsymbol{x} +$
$\epsilon\,\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$

"gibbon"
w/ 99.3%
confidence

Adversarial Examples

- ML models consistently misclassify adversarial examples from dataset
  - such that perturbed input results in model outputting an incorrect answer with high confidence.

  - ML models misclassify examples that are only slightly different from correctly classified examples drawn from data distribution.

  - Adversarial examples are inputs formed by applying small but intentionally worst-case perturbations.

Goodfellow et al. Explaining and Harnessing Adversarial Examples

Adversarial Examples

- ML models such as DNN, clustering, Naive Bayes, Decision tree, Multilayer Perceptron, SVM are not attacked resilient.

- Injection of adversarial/malicious data into training datasets that can caused decreased performance/ model failure is known as poisoning.

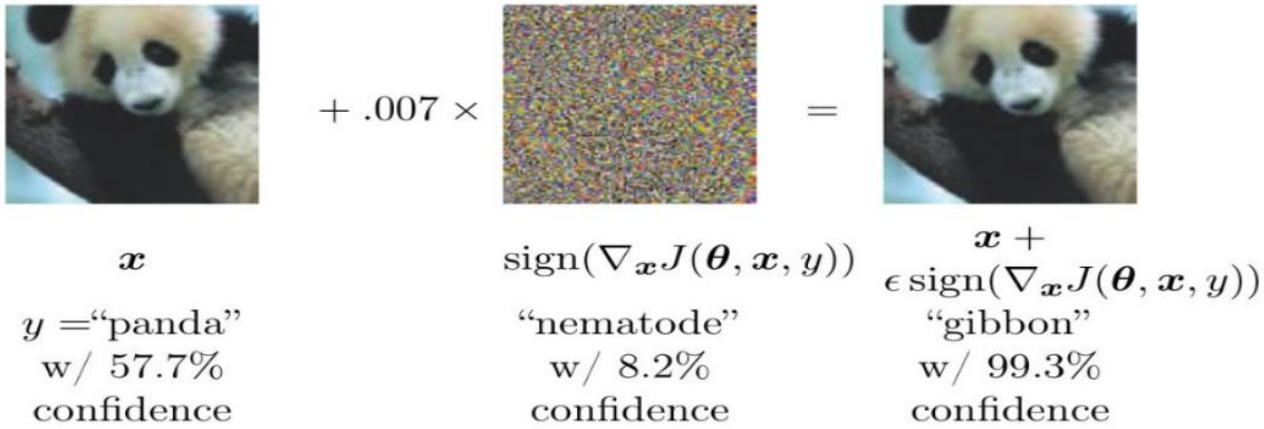
  - Malicious users add malicious data with similar features of original data and wrong labels.

  - Malicious users might know the training data distribution; also learning algorithm.

Goodfellow et al. Explaining and Harnessing Adversarial Examples

Adversarial Examples

- A model is performing a task (e.g. classification) with some level of success.
- An adversary is attacking that model.
- Objective: to perform the task (maintaining a similar accuracy) under attack.



- Forms of adversarial image attacks:
  - Untargeted adversarial attacks: *cannot* control output label of adversarial image.
  - Targeted adversarial attacks: *can* control output label of image.

Goodfellow et al. Explaining and Harnessing Adversarial Examples

Adversarial Examples

- Two types of attacks in DL: White-Box and Black-Box attacks
  - White-Box: Attacker has access to training method (data/network initialization/ algorithm/hyperparameter).
    - Small perturbations --> bad performance.
  - Black-Box attacks: Attacker does not have complete access to network training method.

- We need robust models: weight decay and dropout do not work.
- Approach: Brute force method to generate adversarial examples and train model using them (Adversarial training).

Generative Adversarial Network (GAN)

- A generative adversarial network (GAN) is a deep learning architecture.
- A GAN is called adversarial because it trains two different networks and pits them against each other.
  - One network generates new data by taking an input data sample and modifying it as much as possible.
    - Called Generator network
  - The other network tries to predict whether the generated data output belongs in the original dataset.
    - Called Discriminator network.
      - Attempts to maximize the probability of mistake by the discriminator, but the discriminator attempts to minimize the probability of error.
      - In the equilibrium state, the discriminator can no longer recognize synthesized data.

GAN

- Applications:
  - Generate images
  - Generate training data for other models
  - Complete missing information
  - Generate 3D models from 2D data
- GAN types:
  - Vanilla GAN generates data variation with little or no feedback from the discriminator network. It uses simple MLPs for both generator and discriminator,
  - conditional GAN (cGAN) introduces the concept of conditionality, allowing for targeted data generation.
  - Deep convolutional GAN (DCGAN) integrates CNN architectures into GANs and introduces architectural guidelines to make training more stable.
  - Super-resolution GANS (SRGANs) focus on upscaling low-resolution images to high resolution.

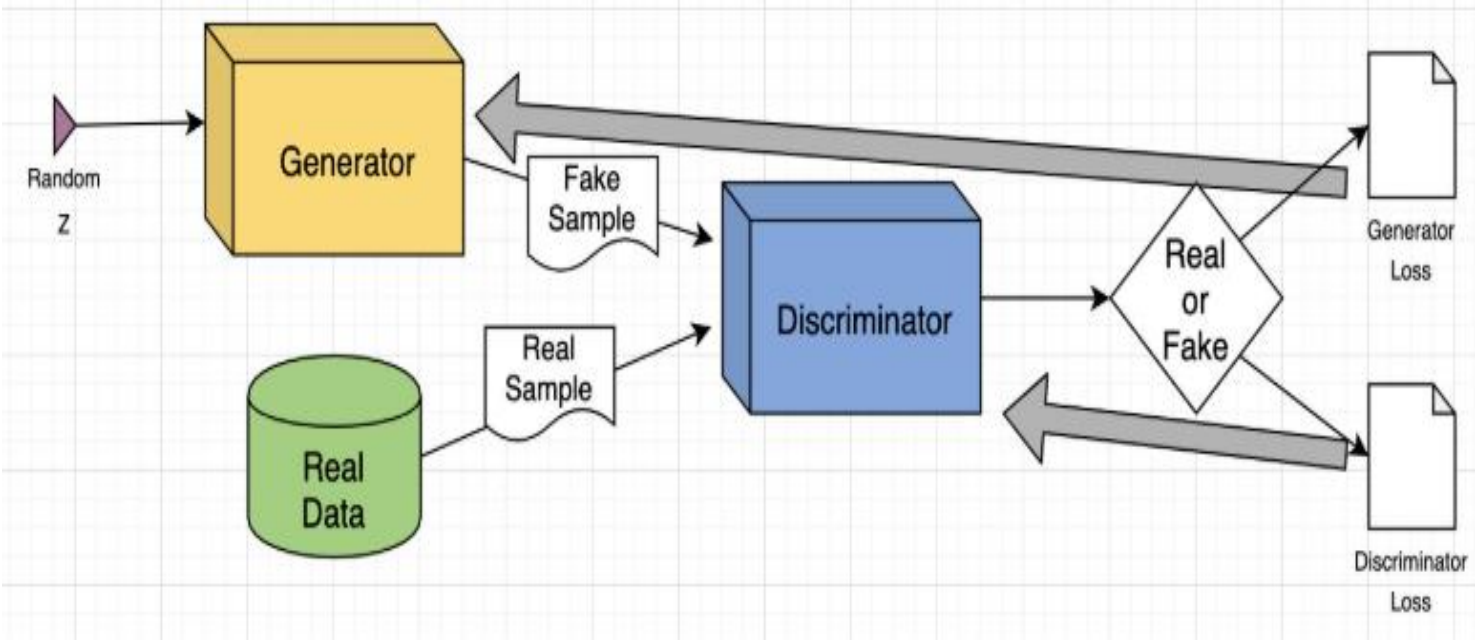
zebra → horse

horse → zebra

https://aws.amazon.com/what-is/gan/

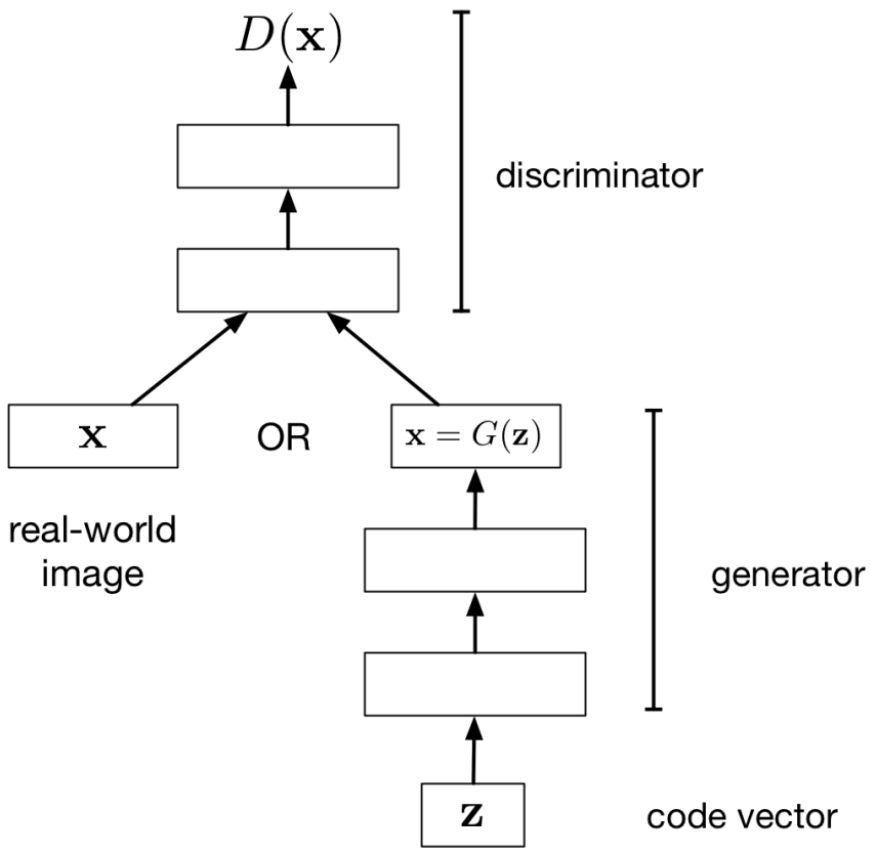
GAN

1. Generator analyzes the training set and identifies data attributes
2. Discriminator analyzes the initial training data and distinguishes between the attributes independently
3. Generator modifies some data attributes by adding noise to certain attributes
4. Generator passes the modified data to the discriminator
5. Discriminator calculates the probability that the generated output belongs to the original dataset
6. Discriminator gives some guidance to the generator to reduce the noise vector randomization in the next cycle.



GAN

- Discriminator evaluates both the generated samples and the data from the training set and decides whether it’s real or fake.
- Assigns a score between 0 and 1
  - 1 : Data looks real
  - 0 : It’s fake.
- Backpropagation is then used to optimize both the networks.
- Gradient of the loss function is calculated according to the network's parameters, and adjusted to minimize the loss.
- Generator uses feedback from the discriminator to improve, trying to create more realistic data.



GAN

- Each training iteration is divided into two phases:
  - Discriminator training: Batch of real images (label 1) is sampled from training set and is completed with an equal number of fake images (label 0) produced by the generator. [Uses binary cross-entropy as loss function].

  - Generator training: Produce another batch of fake images, and once again discriminator is used to tell whether images are fake or real. Do not add real images in the batch, and all labels are set to 1 (real).
    - Never actually sees any real images.

SELU (Scaled Exponential Linear Unit) [2017] aims to automatically normalize the output of neurons during training. [variation of **ReLU**]

GAN

- Network transforms a 30-dimensional random vector into a **28×28 grayscale image**, expanding noise into structured pixel data.

```
codings_size = 30 # 30-dimensional latent vector (random noise).
generator = keras.models.Sequential([ # feed-forward neural network
keras.layers.Dense(100, activation="selu", input_shape=[codings_size]), # Fully connected layer with 100 neurons
keras.layers.Dense(150, activation="selu"), #Fully connected layer with 150 neurons
keras.layers.Dense(28 * 28, activation="sigmoid"), #Outputs 784 neurons (28×28); sigmoid maps values to [0,1]
keras.layers.Reshape([28, 28]) #Reshapes the 784 vector into a 28×28 image
])

discriminator = keras.models.Sequential([ #binary classifier:
keras.layers.Flatten(input_shape=[28, 28]), #Converts 28×28 image → 784-length vector
keras.layers.Dense(150, activation="selu"), #Fully connected layer with 150 neurons; Learns high-level features
keras.layers.Dense(100, activation="selu"), #Further feature abstraction
keras.layers.Dense(1, activation="sigmoid") #Single neuron output
])

gan = keras.models.Sequential([generator, discriminator]) (Noise→Generator→Fake Image→Discriminator→Real/Fake Probability)
```

Pipeline becomes:

1. Random noise (30-d vector)
2. Generator creates 28×28 image
3. Discriminator evaluates that image
4. Output = probability of being real

Adversarial Training

- Models learns to classify correctly adversarial examples.
- Classifiers uses a loss function to minimize model prediction errors.
- After training, attacker uses loss function to maximize model prediction error
  - 1. Compute its gradient with respect to model input
  - 2. Take the sign of gradient and multiply it by a threshold

- Normal (top)
- Noisy (middle)
- Adversarial (bottom)

Adversarial training

- Adversarial training is process of training a model to correctly classify both unmodified examples and adversarial examples.
  - Adv: robust adversarial examples, generalize performance for original examples.
- Virtual adversarial training [VAT]* extends idea of adversarial training to semi-supervised regime and unlabelled examples.
    - VAT uses an efficient approximation to maximize the likelihood of the model while promoting the model's local distributional smoothness on each training input data point.
- Traditional adversarial and virtual adversarial training can be interpreted as regularization strategy and as defence against malicious inputs.

*Takeru Miyato, Shin-ichi Maeda, Masanori Koyama, Ken Nakae, and Shin Ishii. Distributional smoothing with virtual adversarial training. In ICLR, 2016.

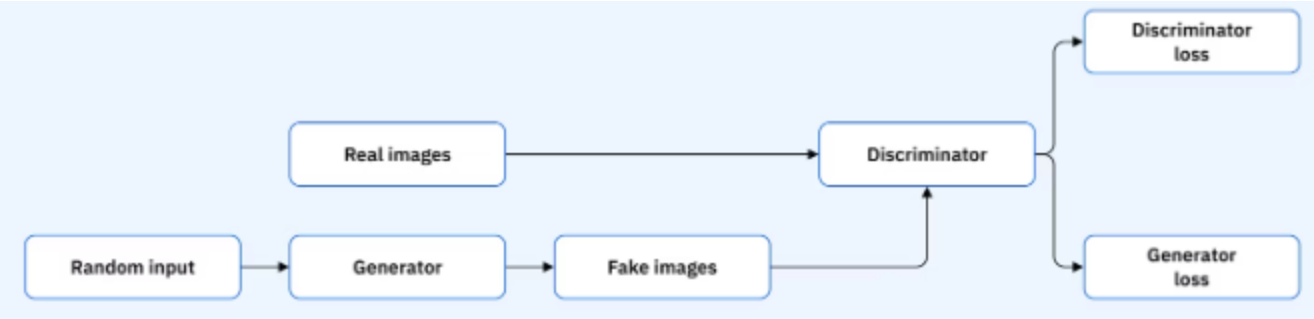
Difficulties of Training GANs

1. Generator and discriminator constantly try to outsmart each other lead to no player would be better off changing their own strategy, assuming other players do not change theirs.

2. Generator produces perfectly realistic images and discriminator is forced to guess (50% real, 50% fake) (Not guaranteed).

3. Generator's outputs gradually become less diverse (called mode collapse).

4. Generator and discriminator are constantly pushing against each other, parameters may end up oscillating and becoming unstable.



How to implement a GAN model? [using Tensorflow and Keras]

- Step 1: Import the required libraries, including TensorFlow and other essential libraries like numpy and matplotlib for building and training the GAN model.
- Step 2: Load and preprocess the dataset, helping ensure it represents the target data distribution (for example, images, text and more).
- Step 3: Build the generator model by using TensorFlow or Keras layers that take random noise and produce data samples matching the target distribution.
- Step 4: Build the discriminator model to classify real vs. fake data samples generated by the generator.
- Step 5: Use suitable optimizers for both generator and discriminator and define loss functions.

How to implement a GAN model? [using Tensorflow and Keras]

- Step 6: Combine the generator and discriminator into a single GAN modelfor training the generator to deceive the discriminator.
- Step 7: Implement a loop to alternate between training the discriminator andthe generator with real and fake data.
- Step 8: Analyze the generator's output and discriminator accuracy overepochs to help ensure convergence.
- Step 9: Use the trained generator to produce new samples that mimic thetarget data distribution.
- Step 10: Plot or analyze the generated data to validate how well the GANhas learned the target distribution.

https://www.ibm.com/think/topics/generative-adversarial-networks

Summary

- GANs are a framework for training generative models.
- These models learn to map random noise vectors to outputs that resemble a particular data distribution, such as images.

- Core idea: To have two neural networks compete against each other:
  - Generator Network: This network takes a random distribution (typically Gaussian) as input and tries to produce realistic-looking samples, such as images.
  - Discriminator Network: This network takes either a fake image from the generator or a real image from the training set as input.

- Each training iteration in a GAN is divided into two phases:
  - Discriminator Training: A batch of real images (labelled as 1) is sampled from the training set and combined with an equal number of fake images (labelled as 0) produced by the generator.
  - Generator Training: The generator produces another batch of fake images. The discriminator is then used to evaluate these fake images, but this time all the labels are set to 1 (real).

- The constant competition between the generator and discriminator can lead to a situation where neither network improves significantly, and the process does not converge to a stable equilibrium.